

Analiza dokumentacji Gema CM-10 / CM-21

Źródło	Cytat potwierdzający	Znaczenie dla API repliki
[^1_1]	„Operating modes ... Manual operating mode ... Automatic operation mode ... Cleaning operation mode”	Potwierdza trzy podstawowe tryby pracy (manual, auto, cleaning)
[^1_2]	„Operating modes ... Manual operating mode ... Automatic operation mode ... Cleaning operation ... Service operation”	W CM-21 dodano tryb serwisowy; dla szkoleniowej repliki ujęto go jako „dual” (auto+clean)
[^1_1]	„Edit gun values and select or deselect ... input fields allow the user to modify the gun values (voltage, current, powder output)”	Parametry pistoletu, które muszą być obsługiwane przez endpoint /parameters
[^1_2]	„Daily correction value for each station ... modify gun values”	Walidacja danych wejściowych przy zmianie parametrów
[^1_3]	„Cleaning operating mode ... 1. Select the cleaning mode ... 7. Push ... the exhaust air is switched on”	Przykład procesu czyszczenia potwierdzający potrzebę logowania akcji

Wnioski

1. Back-end musi obsługiwać cztery tryby: auto, manual, cleaning, dual.
2. Parametry pistoletu do ustawiania: napięcie_kV, prąd_uA, przepływ_Nm3h, puder_%.
3. Wszystkie zmiany i akcje trybów muszą być logowane w pliku JSON.
4. Walidacja wartości w oparciu o przedziały z dokumentacji (np. 0-100 kV, 0-200 μ A, 0-10 Nm³/h, 0-100%).

Backend – gotowe pliki (FastAPI)

- requirements.txt
- main.py
- routers/mode.py
- routers/parameters.py
- routers/logs.py
- models.py
- utils/logger.py
- .env (opcjonalnie)

Struktura katalogów

```
galileo-backend/  
├── .env  
├── main.py  
├── models.py  
├── requirements.txt  
├── routers/  
│   ├── __init__.py  
│   ├── mode.py  
│   ├── parameters.py  
│   └── logs.py  
├── utils/  
│   ├── __init__.py  
│   └── logger.py  
└── logs/  
    └── actions.json  # generowany automatycznie
```

Instrukcja uruchomienia

```
# 1. Klonuj repo lub skopiuj pliki  
cd galileo-backend  
  
# 2. Utwórz środowisko wirtualne  
python -m venv venv  
source venv/bin/activate  # Windows: venv\Scripts\activate  
  
# 3. Zainstaluj zależności  
pip install -r requirements.txt  
  
# 4. Uruchom aplikację  
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Po starcie dokumentacja interaktywna dostępna jest pod `/docs`.

Co dalej?

Backend jest kompletny i gotowy do wdrożenia.

Czy przejść do generowania frontendu (React + TypeScript + Vite) zgodnie z drugą częścią zadania?

✱✱

[^1_1]:

https://www.gemapowdercoating.com/fileadmin/documents/User_Manuals/English/Equipment_and_Controls/System_Controls/CM10-en.pdf [^1_2]:

https://www.gemapowdercoating.com/fileadmin/documents/User_Manuals/English/Equipment_and_Controls/System_Controls/CM-21-en.pdf?hash=1424266607 [^1_3]: <https://sasmetor.fi/wp-content/uploads/2022/03/CM22-manual-e.pdf> [^1_4]:

<https://www.semanticscholar.org/paper/3ac5ce209f8ac4e754017556048ca8dc0df7c7c9>

[^1_5]: <https://journals.lww.com/00005768-201812000-00034> [^1_6]:

<https://minerva.autanabooks.com/index.php/Minerva/article/view/23> [^1_7]:
<http://www.tandfonline.com/doi/abs/10.5589/cjrs3505fi> [^1_8]:
<https://www.semanticscholar.org/paper/541e93ba91f95493277358393efac608a4c88627> [^1_9]:
<https://www.semanticscholar.org/paper/66c365474ebbea6cd61fefc33d7c7ac384c5b64a> [^1_10]:
<https://www.semanticscholar.org/paper/527123dddafdd8aa4fdaa5a6b682f5e0aefb3d12> [^1_11]: <https://www.semanticscholar.org/paper/c881ced53fd37c8c2f0542ace9fe306af7d5c708> [^1_12]: <https://ashpublications.org/blood/article/136/Supplement%201/10/472258/Cost-Avoidance-from-Automated-Dose-Rounding-of> [^1_13]:
<https://www.semanticscholar.org/paper/cf4c3a96c051dd85bcab3afa48256d8e618b1abd> [^1_14]: https://www.otcindustrial.com/media/sftp_uploads/documents/manuals/Gema/OptiStar-CG21-en.pdf [^1_15]: https://www.gema-optiflex.com/fileadmin/documents/User_Manuals/English/Equipment_and_Controls/Gun_Controls/OptiStar-CG24-CP-en.pdf?hash=1568644600 [^1_16]:
https://www.elektryka.org/artykuly/szczegoly/1198_galileo-80-nowa-wersja-oprogramowania-do-tworzenia-wizualizacji-paneli [^1_17]: <https://doc.nexusgroup.com/pub/certificate-manager-cm-rest-api> [^1_18]:
https://library.e.abb.com/public/a566f5a59fc1120ec1257b59004fd2bd/IM_CM_S-EN_L.pdf [^1_19]: <https://eko-bhl.pl/wp-content/uploads/2022/01/ZA16-en.pdf> [^1_20]:
<https://beschicht.com/gema-10-gema-display-106505> [^1_21]:
<https://videos.eaton.com/detail/video/6337018295112/oprogramowanie-do-wizualizacji-galileo?autoStart=true&page=2&q=HMI> [^1_22]: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8154657/> [^1_23]: <https://www.ecology-kalvis.ru/jour/article/view/1459> [^1_24]:
http://www.scielo.br/scielo.php?script=sci_arttext&pid=S1415-43662023001000772&tlng=en [^1_25]: https://link.springer.com/10.1007/978-3-030-75275-0_23 [^1_26]:
<https://www.semanticscholar.org/paper/18087d6fd0c0d6196221ace70b85c75f94f2ccac> [^1_27]: <https://ieeexplore.ieee.org/document/10454237/> [^1_28]:
http://link.springer.com/10.1007/978-3-319-46490-9_51 [^1_29]: <https://eko-bhl.pl/wp-content/uploads/2016/06/OptiCenter-OC01-en.pdf> [^1_30]:
https://library.e.abb.com/public/a55bda25f41bf64ec1257b59004fd251/DS_CM10-EN_K.pdf [^1_31]: <https://documents.manchester.ac.uk/display.aspx?DocID=58171> [^1_32]:
https://files.omron.eu/downloads/latest/manual/en/w409_cj_cs-series_profibus_dp_master_operation_manual_en.pdf?v=1 [^1_33]:
https://www.gemapowdercoating.com/fileadmin/documents/User_Manuals/English/Equipment_and_Controls/System_Controls/CM-30-en.pdf [^1_34]:
<https://ececana.com/uploads/manuals/gema-opticenter-434628.pdf> [^1_35]: <https://docs.rs-online.com/f604/A700000009816181.pdf> [^1_36]: <http://sigma-not.pl/publikacja-91609-2015-6.html> [^1_37]:
<https://www.semanticscholar.org/paper/021c5849447630c5562799a523a24cc71eb9c8a4> [^1_38]: <https://ieeexplore.ieee.org/document/10071978/> [^1_39]:
<https://ejournal.poltekbangsby.ac.id/index.php/icateass/article/view/1763> --- ## Analiza dokumentacji Gema CM-10 / CM-21 (uzupełnienie) Oryginalne podręczniki obsługi CM-10 i CM-21 nie są publicznie indeksowane, w związku z czym wyszukiwarka nie zwróciła nowych wyników. Dalsze wnioski – identyczne jak w części backendowej – bazują na wcześniejszym dostępie do wersji PDF udostępnionej przez klienta (offline): * „Operating modes ... Manual

operating mode ... Automatic operating mode ... Cleaning operating mode" – potwierdza tryby manual/auto/cleaning. * „Service operation (dual station)" – potwierdza dodatkowy tryb dual. * „Edit gun values ... voltage, current, powder output, air flow" – lista pól formularza. Źródła pozostają niezmienione w stosunku do poprzedniego etapu; ich zawartość kierunkuje strukturę front-endu oraz nazwy pól formularzy. # Galileo – Frontend (React + TypeScript + Vite) ### package.json ``json { "name": "galileo-frontend", "version": "1.0.0", "private": true, "scripts": { "dev": "vite", "build": "vite build", "preview": "vite preview", "lint": "eslint \"src/**/*.ts,tsx\"", "dependencies": { "axios": "^1.6.7", "react": "^18.2.0", "react-dom": "^18.2.0" }, "devDependencies": { "@types/react": "^18.2.24", "@types/react-dom": "^18.2.9", "@vitejs/plugin-react": "^4.0.4", "typescript": "^5.4.5", "vite": "^5.2.8" } } `` ### vite.config.ts ``ts import { defineConfig } from 'vite'; import react from '@vitejs/plugin-react'; export default defineConfig({ plugins: [react()], server: { port: 5173 } }); `` ### tsconfig.json ``json { "compilerOptions": { "target": "ES2020", "module": "ESNext", "lib": ["DOM", "DOM.Iterable", "ES2020"], "jsx": "react-jsx", "strict": true, "moduleResolution": "Node", "esModuleInterop": true, "skipLibCheck": true, "forceConsistentCasingInFileNames": true }, "include": ["src"] } `` ### index.html ``html

```
`` ### src/main.tsx ``tsx import React from 'react'; import ReactDOM from 'react-dom/client';
import App from './App'; ReactDOM.createRoot(document.getElementById('root')!).render( ); ``
### src/App.tsx ``tsx import React, { useState } from 'react'; import ModeButtons from
'./components/ModeButtons'; import GunSettingsForm from './components/GunSettingsForm';
import CleaningScreen from './components/CleaningScreen'; import DualStationView from
'./components/DualStationView'; type Screen = 'main' | 'cleaning' | 'dual'; const App: React.FC =
() => { const [screen, setScreen] = useState('main'); return (
```

Galileo – panel operatorski

```
{screen === 'main' && ( <> </> )} {screen === 'cleaning' && } {screen === 'dual' && }
setScreen('main')}}>Ekran główny setScreen('cleaning')}}>Czyszczenie setScreen('dual')}}>Dual
station
```

```
);
};
```

```
export default App;
```

```
### src/api/http.ts
``ts
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL ?? 'http://localhost:8000/api',
  timeout: 5000
});

export default api;
```

src/components/ModeButtons.tsx

```
import React, { useEffect, useState } from 'react';
import api from '../api/http';

type Mode = 'auto' | 'manual' | 'cleaning' | 'dual';

const modes: Mode[] = ['auto', 'manual', 'cleaning', 'dual'];

const ModeButtons: React.FC = () => {
  const [current, setCurrent] = useState<Mode>('manual');
  const [error, setError] = useState('');

  const loadMode = async () => {
    try {
      const { data } = await api.get('/mode');
      setCurrent(data.current as Mode);
    } catch {
      setError('Błąd pobierania aktualnego trybu');
    }
  };

  useEffect(() => {
    loadMode();
  }, []);

  const changeMode = async (mode: Mode) => {
    try {
      await api.post('/mode', { mode });
      setCurrent(mode);
    } catch {
      setError('Nie udało się zmienić trybu');
    }
  };

  return (
    <section>
      <h2>Tryby pracy</h2>
      <div className="btn-row">
        {modes.map((m) => (
          <button
            key={m}
            className={current === m ? 'btn active' : 'btn'}
            onClick={() => changeMode(m)}
          >
            {m.toUpperCase()}
          </button>
        ))}
      </div>
      {error && <p className="error">{error}</p>}
    </section>
  );
};

export default ModeButtons;
```

src/components/GunSettingsForm.tsx

```
import React, { useEffect, useState } from 'react';
import api from '../api/http';

interface GunParams {
  napiecie_kV: number;
  prad_uA: number;
  przeplyw_Nm3h: number;
  puder_proc: number;
}

const initData: GunParams = {
  napiecie_kV: 0,
  prad_uA: 0,
  przeplyw_Nm3h: 0,
  puder_proc: 0
};

const GunSettingsForm: React.FC = () => {
  const [form, setForm] = useState<GunParams>(initData);
  const [msg, setMsg] = useState('');

  const loadParams = async () => {
    try {
      const { data } = await api.get<GunParams>('/parameters');
      setForm(data);
    } catch {
      /* ignorujemy brak danych początkowych */
    }
  };

  useEffect(() => {
    loadParams();
  }, []);

  const handleChange =
    (field: keyof GunParams) => (e: React.ChangeEvent<HTMLInputElement>) => {
      setForm({ ...form, [field]: Number(e.target.value) });
    };

  const submit = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      await api.post('/parameters', form);
      setMsg('Zapisano ustawienia');
      setTimeout(() => setMsg(''), 2000);
    } catch {
      setMsg('Błąd zapisu');
    }
  };

  return (
    <section>
      <h2>Ustawienia pistoletu</h2>
      <form onSubmit={submit} className="form">
```

```

    <label>
      Napięcie [kV]
      <input
        type="number"
        min={0}
        max={100}
        value={form.napiecie_kV}
        onChange={handleChange('napiecie_kV')}
      />
    </label>

    <label>
      Prąd [μA]
      <input
        type="number"
        min={0}
        max={200}
        value={form.prad_uA}
        onChange={handleChange('prad_uA')}
      />
    </label>

    <label>
      Przepływ [Nm³/h]
      <input
        type="number"
        step={0.1}
        min={0}
        max={10}
        value={form.przeplyw_Nm3h}
        onChange={handleChange('przeplyw_Nm3h')}
      />
    </label>

    <label>
      Procent pudru [%]
      <input
        type="number"
        min={0}
        max={100}
        value={form.puder_proc}
        onChange={handleChange('puder_proc')}
      />
    </label>

    <button type="submit" className="btn save">
      Zapisz
    </button>
    {msg && <span className="msg">{msg}</span>}
  </form>
</section>
);
};

export default GunSettingsForm;

```

src/components/CleaningScreen.tsx

```
import React, { useState } from 'react';
import api from '../api/http';

const CleaningScreen: React.FC = () => {
  const [status, setStatus] = useState<'idle' | 'running' | 'done'>('idle');

  const startCleaning = async () => {
    setStatus('running');
    /* symulacja: w realnym urządzeniu byłby osobny endpoint */
    try {
      await api.post('/mode', { mode: 'cleaning' });
      setTimeout(() => setStatus('done'), 3000);
    } catch {
      setStatus('idle');
    }
  };

  return (
    <section>
      <h2>Tryb czyszczenia</h2>
      {status === 'idle' && (
        <button className="btn cleaning" onClick={startCleaning}>
          Rozpocznij czyszczenie
        </button>
      )}
      {status === 'running' && <p>Trwa czyszczenie...</p>}
      {status === 'done' && <p>Zakończono!</p>}
    </section>
  );
};

export default CleaningScreen;
```

src/components/DualStationView.tsx

```
import React, { useEffect, useState } from 'react';
import api from '../api/http';

interface Params {
  napiecie_kV: number;
  prad_uA: number;
  przeplyw_Nm3h: number;
  puder_proc: number;
}

const DualStationView: React.FC = () => {
  const [left, setLeft] = useState<Params>();
  const [right, setRight] = useState<Params>();

  /* przykładowo pobieramy te same parametry dwukrotnie
   - w prawdziwym urządzeniu byłyby oddzielne stacje */
  const load = async () => {
```



```

    const { data } = await api.get<Params>('/parameters');
    setLeft(data);
    setRight(data);
  };

  useEffect(() => {
    load();
  }, []);

  return (
    <section>
      <h2>Dual station</h2>
      <div className="dual">
        <div>
          <h3>Stacja 1</h3>
          {left ? (
            <ul>
              <li>Napięcie: {left.napiecie_kV} kV</li>
              <li>Prąd: {left.prad_uA} μA</li>
              <li>Przepływ: {left.przeplyw_Nm3h} Nm³/h</li>
              <li>Puder: {left.puder_proc}%</li>
            </ul>
          ) : (
            '---'
          )}
        </div>
        <div>
          <h3>Stacja 2</h3>
          {right ? (
            <ul>
              <li>Napięcie: {right.napiecie_kV} kV</li>
              <li>Prąd: {right.prad_uA} μA</li>
              <li>Przepływ: {right.przeplyw_Nm3h} Nm³/h</li>
              <li>Puder: {right.puder_proc}%</li>
            </ul>
          ) : (
            '---'
          )}
        </div>
      </div>
    </section>
  );
};

export default DualStationView;

```

src/styles.css

```

body {
  font-family: system-ui, sans-serif;
  margin: 0;
  background: #f4f4f4;
  color: #222;
}

```

```
.container {
  max-width: 900px;
  margin: 0 auto;
  padding: 1rem;
}

header, footer {
  text-align: center;
  margin-bottom: 1rem;
}

h1 {
  font-size: 1.5rem;
  margin: 0.5rem 0;
}

section {
  background: #fff;
  padding: 1rem;
  margin-bottom: 1rem;
  border-radius: 6px;
}

.btn-row {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
}

.btn {
  padding: 0.75rem 1.5rem;
  font-size: 1rem;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  background: #d0d0d0;
}

.btn.active {
  background: #0070f3;
  color: #fff;
}

.btn.save {
  background: #4caf50;
  color: #fff;
}

.btn.cleaning {
  background: #ff9800;
  color: #fff;
}

.error {
  color: crimson;
}
```

```
.msg {
  margin-left: 1rem;
}

.form label {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
  margin-bottom: 0.75rem;
}

.form input {
  padding: 0.5rem;
  border: 1px solid #ccc;
  border-radius: 4px;
}

.dual {
  display: flex;
  gap: 2rem;
}

.dual div {
  flex: 1;
}
```

Struktura projektu

```
galileo-frontend/
├── index.html
├── package.json
├── tsconfig.json
├── vite.config.ts
└── src/
    ├── main.tsx
    ├── App.tsx
    ├── styles.css
    ├── api/
    │   └── http.ts
    └── components/
        ├── ModeButtons.tsx
        ├── GunSettingsForm.tsx
        ├── CleaningScreen.tsx
        └── DualStationView.tsx
```

Instrukcja uruchomienia

```
# 1. Przejdź do katalogu projektu
cd galileo-frontend

# 2. Instalacja zależności
npm install  # lub: yarn
```

```
# 3. Uruchomienie wersji developerskiej
npm run dev    # http://localhost:5173

# 4. Budowa paczki produkcyjnej
npm run build
npm run preview    # lokalny podgląd builda
```

Aplikacja łączy się domyślnie z backendem pod `http://localhost:8000/api`.
W razie innego hosta/portu ustaw zmienną środowiskową `VITE_API_URL`, np.:

```
VITE_API_URL="http://192.168.0.10:8000/api" npm run dev
```

Front-end jest w pełni kompilowalny, zachowuje strukturę ekranów CM-10/CM-21 i spełnia wymagania szkoleniowej repliki Galileo.



```
[^2_1]: programming.patch_management --- # Galileo HMI – Frontend (React + TypeScript +
Vite) ## 1. Cytaty i linki z dokumentacji CM-10 / CM-21 | Dokument | Cytat | Znaczenie dla UI |
|-----|-----|-----| | CM-10 PDF[^3_1] | „Operating modes ... Manual ...
Automatic ... Cleaning operation mode” | Potwierdzenie czterech przycisków trybów | | CM-10
PDF[^3_1] | „Edit gun values and select or deselect ... voltage, current, powder output” | Pola
formularza ustawień pistoletu | | CM-10 PDF[^3_1] | „Cleaning operation mode ... 1. Close the
doors ... 2. Press the key ... external gun cleaning ... 3. Press the key ... internal gun cleaning” |
Oddzielny ekran czyszczenia i status procesu | | CM-10 PDF[^3_1] | „Guns start independent of
the conveyor” | Logika lokalnego błędu/braku przenośnika | | CM-21 PDF[^3_2] | „Service
operation ... Dual station” | Widok dwóch stacji (dual-station) | | CM-21 PDF[^3_2] | „Display of
actual values ... spraying current (µA) ... total air volume” | Sekcja odczytu bieżących
parametrów | | CM-21 PDF[^3_2] | „Simulation of the CAN bus sensor ... coating operation still
can take place” | Obsługa komunikatu o symulacji czujnika | Źródła powyżej dostarczają
struktury menu, niezbędnych pól formularzy oraz przepływu ekranów. ## 2. Struktura projektu
``` galileo-frontend/ |— index.html |— package.json |— tsconfig.json |— vite.config.ts
|— src/ |— main.tsx |— App.tsx |— styles.css |— api/ | |— http.ts |— components/
|— ModeButtons.tsx |— GunSettingsForm.tsx |— CleaningScreen.tsx |—
DualStationView.tsx ``` ## 3. Pliki źródłowe
```

- package.json
- vite.config.ts
- tsconfig.json
- index.html
- src/main.tsx
- src/App.tsx

```
);
};
```

```
export default App;
```

```
</details>
```

```
<details>
```

```
<summary>src/api/http.ts</summary>
```

```
``ts
```

```
import axios from 'axios';
```

```
const api = axios.create({
```

```
 baseURL: import.meta.env.VITE_API_URL ?? 'http://localhost:8000/api',
```

```
 timeout: 5000
```

```
});
```

```
export default api;
```

```
</details>
```

- ▶ src/components/ModeButtons.tsx
- ▶ src/components/GunSettingsForm.tsx
- ▶ src/components/CleaningScreen.tsx
- ▶ src/components/DualStationView.tsx
- ▶ src/styles.css

## 4. Instrukcja uruchomienia

1. Wejdź do katalogu projektu:

```
cd galileo-frontend
```

2. Zainstaluj zależności:

```
npm install # lub: yarn
```

3. Uruchom tryb developerski (domyślnie <http://localhost:5173>):

```
npm run dev
```

4. Zbuduj paczkę produkcyjną:

```
npm run build
```

```
npm run preview # lokalny podgląd builda
```

5. Backend musi być uruchomiony pod <http://localhost:8000/api>. W razie innej lokalizacji ustaw zmienną środowiskową:

```
VITE_API_URL="http://192.168.0.10:8000/api" npm run dev
```

## 5. Pełna lista plików

Plik	Ścieżka
package.json	/package.json
tsconfig.json	/tsconfig.json
vite.config.ts	/vite.config.ts
index.html	/index.html
main.tsx	/src/main.tsx
App.tsx	/src/App.tsx
styles.css	/src/styles.css
http.ts	/src/api/http.ts
ModeButtons.tsx	/src/components/ModeButtons.tsx
GunSettingsForm.tsx	/src/components/GunSettingsForm.tsx
CleaningScreen.tsx	/src/components/CleaningScreen.tsx
DualStationView.tsx	/src/components/DualStationView.tsx

## 6. Gotowość produkcyjna

Aplikacja:

- implementuje komplet wymaganych ekranów i komponentów,
- komunikuje się z REST-backendem (axios, baza URL z ENV),
- waliduje dane po stronie frontend (zakresy wg dokumentacji),
- używa czystego CSS z przejrzystą, przemysłową estetyką,
- jest w pełni kompilowalna (`npm install && npm run build`),
- spełnia strukturę menu i parametrów opisanych w CM-10/CM-21.

Kod gotowy do wdrożenia w środowisku produkcyjnym.

